



CabRoster

The office cab problem, explained simply

A beginner's walk-through before you write any code.
We look at the people, the data, the rules and the clock.

The story

A company gives its employees a free cab to and from the office

Today it is run with spreadsheets and phone calls. That causes three everyday problems:



Over-filled cabs

Nobody checks how many seats are left, so 6 people get told to take a 4-seat cab.



Missed pickups

The driver has an old list. Someone who booked is never picked up.



No record

Nobody knows who actually travelled, who cancelled and who did not show up.

Our job: build the backend (the server part) that fixes all three.

Only two trips a day

Every working day (Monday to Friday) has exactly two service slots



PICKUP

Cab leaves at 06:30



Home



Office

Morning: home → office



DROP

Cab leaves at 16:30



Office



Home

Evening: office → home

Three kinds of users

Each person logs in with a role, and the role decides what they may do



EMPLOYEE

- Signs up by themselves
- Sets home address and zone
- Books a seat, or cancels it
- Sees their own bookings only



ADMIN

- The transport desk
- Adds cabs and drivers
- Creates trips (cab + driver)
- Puts bookings into trips



DRIVER

- Created by the admin
- Sees only their own trips
- Starts the trip
- Marks who got in, then completes

Four things we store

Everything in the system is one of these four tables

users

Everyone who can log in

Name, e-mail, phone, role, home address, zone

cabs

The vehicles

Registration number (KA01AB1234), model, number of seats

bookings

A request for ONE seat

"Priya wants a seat on Monday's PICKUP"

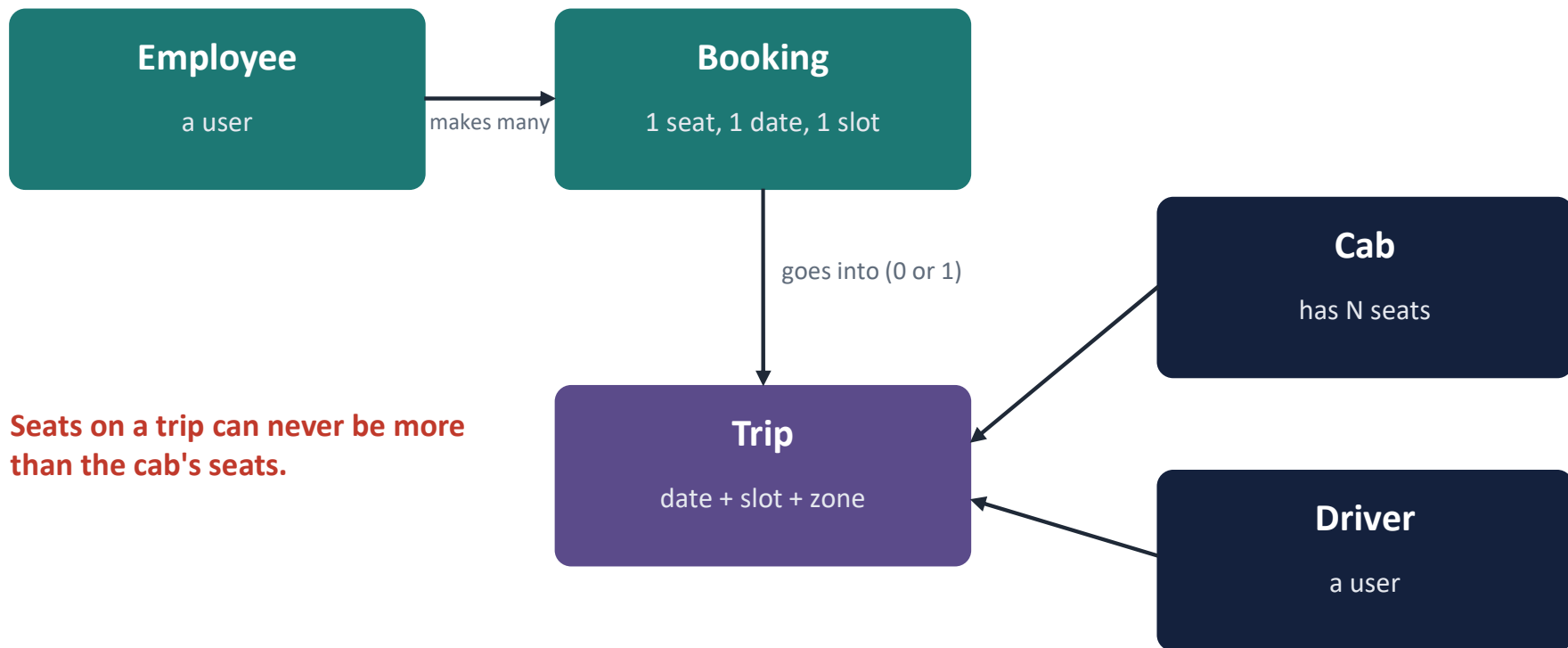
trips

ONE cab run

"Cab KA01AB1234 with driver Ravi, Monday PICKUP, EAST zone"

How the four connect

A trip is a cab + a driver + a list of bookings



Seats on a trip can never be more than the cab's seats.

One Monday, step by step

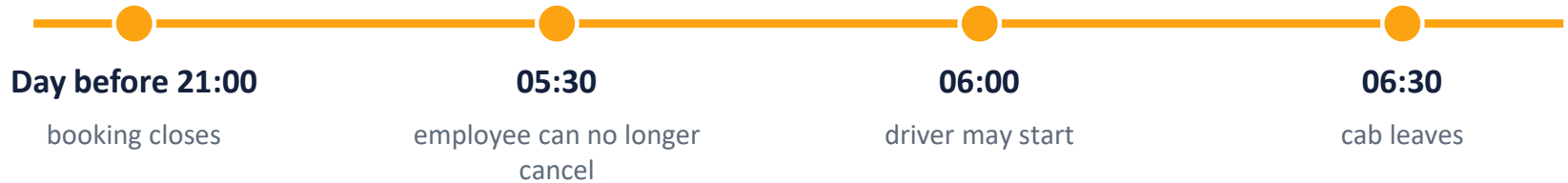
The whole system in six moves

1	Friday 10:00	Priya books Monday PICKUP	EMPLOYEE
2	Friday 17:00	Admin creates a trip: cab + driver + EAST	ADMIN
3	Friday 17:05	Admin puts 4 EAST bookings into the trip	ADMIN
4	Monday 06:05	Driver Ravi starts the trip	DRIVER
5	Monday 06:20	Ravi marks Priya BOARDED (someone else: NO_SHOW)	DRIVER
6	Monday 09:10	Ravi completes the trip. Bookings are closed.	DRIVER

The clock decides a lot

Every deadline, on one line. Exactly at the deadline counts as too late.

PICKUP



DROP



Example: booking a Monday PICKUP at Sunday 20:59 works. At Sunday 21:00:00 it is refused.

The main rules in plain English

Each one has an ID (BR-xx) in the spec. Use the ID in your code comments.

BR-01

Weekdays only

No bookings or trips on Saturday or Sunday.

BR-02

Up to 7 days ahead

Not in the past and not more than a week away.

BR-03

One seat per slot

You can't book Monday PICKUP twice (unless you cancelled the first one).

BR-05

Profile first

You need a home address and zone before you can book.

BR-09

Never over-fill

Passengers on a trip $\leq$ seats in the cab.

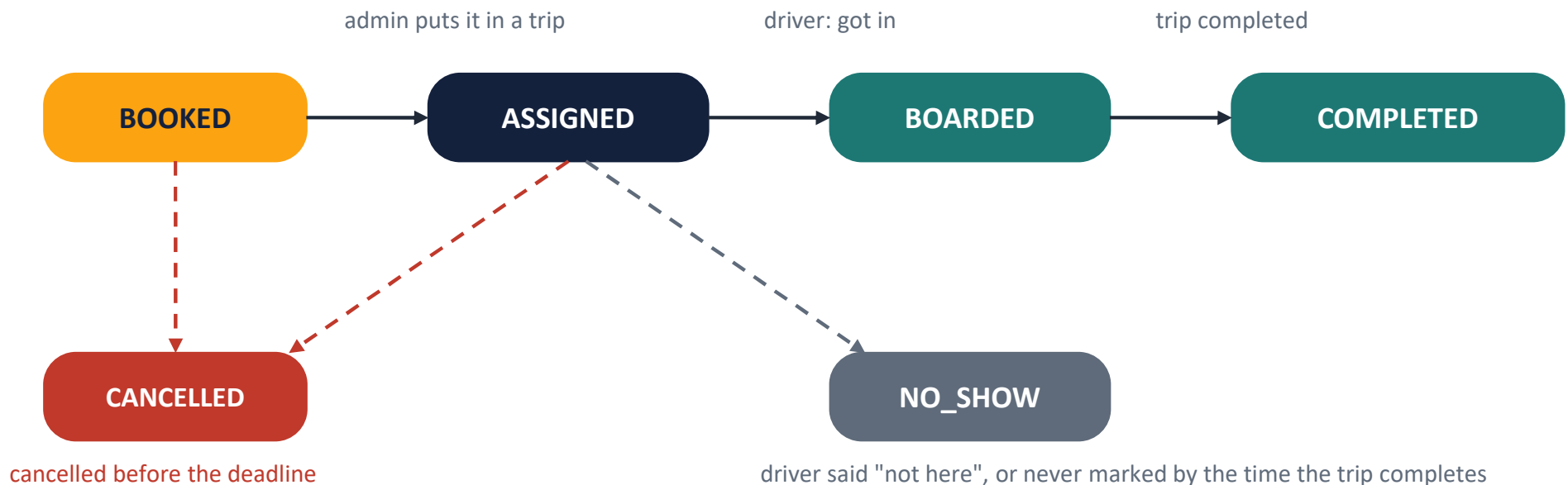
BR-12

Drivers see only their trips

Another driver's trip looks like it does not exist.

The life of a booking

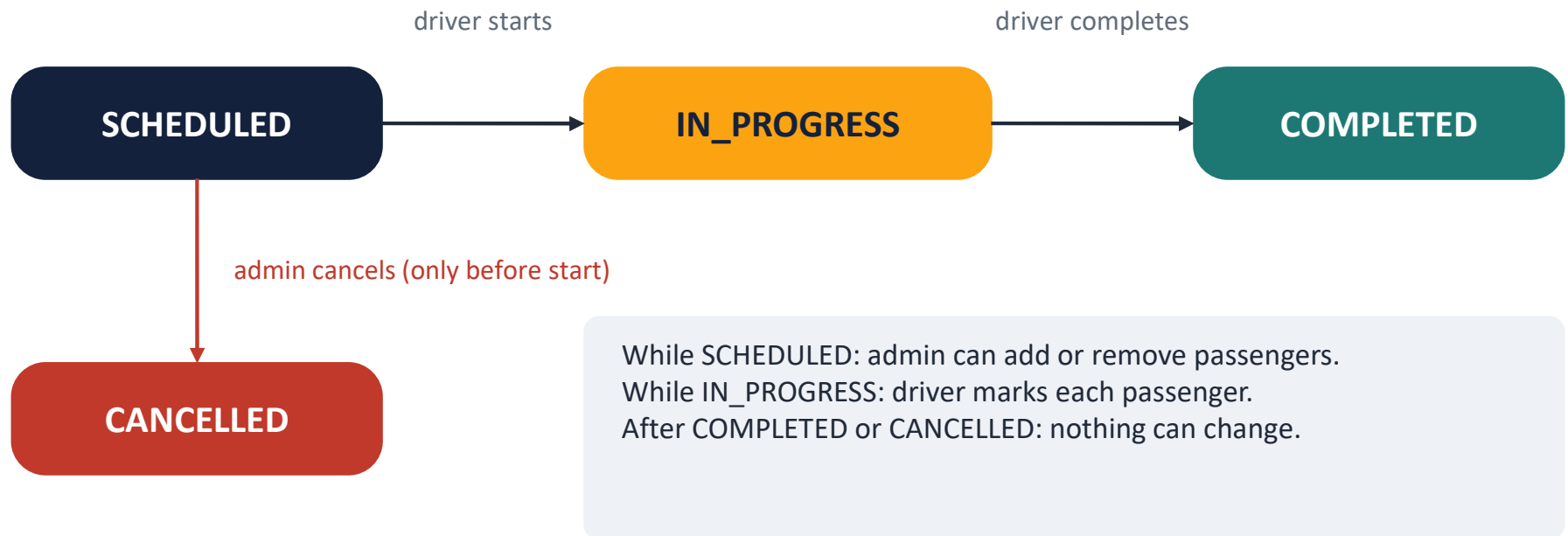
A booking is always in exactly one status



If the admin removes it from a trip or cancels the trip, it goes back to BOOKED.

The life of a trip

Only three moves are allowed. Anything else is an error (409).



What does "backend API" mean?

A client sends a request. Your code answers with a status code and JSON.

Request

```
POST /api/v1/bookings
Authorization: Bearer <token>

{
  "travel_date": "2026-09-28",
  "slot": "PICKUP"
}
```



Response 201 Created

```
{
  "id": 41,
  "slot": "PICKUP",
  "zone": "EAST",
  "status": "BOOKED",
  "trip": null
}
```

Status codes in plain words

Picking the right one is part of the grade

Code	Meaning	CabRoster example
200 / 201	OK / created	Booking saved
401	Who are you? (no token, bad token, bad password)	Calling /bookings without logging in
403	I know you, but your role is not allowed	An employee opening the users list
404	Not found, or it is not yours	Driver B opening driver A's trip
409	Clashes with what already exists or its current state	Booking the same slot twice, a full cab
422	Your input breaks a rule	Saturday booking, booking after the deadline

Logging in = getting a visitor badge

JWT tokens, without the jargon



Log in once

Send e-mail + password to /auth/login. If they are right, you get a token (the badge).



Show the badge every time

Every later request carries the header `Authorization: Bearer <token>`.



Badges expire

After 60 minutes the token stops working → 401. Log in again.



Deactivated = badge revoked

If the admin deactivates you, even your old token gets 401.

Long lists: filter, sort, paginate

Like finding a book in a library

1 Filter

*"Only EAST zone, only Monday
PICKUP"*

`?zone=EAST&slot=PICKUP`

2 Sort

"Newest first"

`?sort_by=travel_date&order=desc`

3 Paginate

*"Show 10 per page, give me page
2"*

`?page=2&size=10`

Every list answers in the same shape:

```
{ "items": [...], "total": 57, "page": 2, "size": 10, "pages": 6 }
```

What you get, and how to work

Starter kit + tests. Fill the TODOs, run pytest, repeat.

- Every route, path and parameter is already declared
- Enums, the User model, config, database and `get_now` are done
- Search for TODO: each one names its backlog item (DEV-xx)
- `tests/simple` has 38 tests, 1–2 per API

Start with `security.py` and `deps.py`. Nothing else passes until login works.

Your loop

1. pick the next TODO
2. write the code
3. `pytest -x`
4. read the first failure
5. fix it, then go back to 1

Five beginner traps

Each of these fails many tests at once



Using `datetime.now()` directly

Use `now = Depends(get_now)`. The tests fake the clock.



Deleting rows

Never delete. Cancel = change the status. Deactivate = `is_active` false.



Returning 403 for someone else's booking

Return 404 so no one can guess which IDs exist.



Saving half of a trip assignment

Check every booking first, then save them all, or save none.



Sorting by any field the user sends

Only allow the fields on the whitelist, anything else → 422.

Before you open the code, can you answer these?

- 1 What is the difference between a booking and a trip?
- 2 Why can't an employee book Monday PICKUP on Sunday at 21:30?
- 3 What happens to bookings when the admin cancels a trip?
- 4 Which status code for a 5th passenger in a 4-seat cab?
- 5 Why does a driver get 404 (not 403) on another driver's trip?